

Assignment 08: RegEx and NLP with SpaCy

Name: Mahesh Janaranjana **Index No:** 20260064 **Programme:** MSc in Applied Artificial Intelligence (IIT) **Module:** RegEx and NLP with SpaCy

This notebook works through five short natural language processing tasks. I start with a regular expression for international phone numbers, then move into spaCy: inspecting token attributes, writing a rule based `Matcher`, running named entity recognition and probing where it breaks, and finally a small hybrid function that combines a regex with dependency parsing. All of the input text is taken directly from the brief. I use the `en_core_web_sm` model throughout.

Setup

One import cell and a single loaded pipeline that every task reuses. I print the versions so the run is reproducible.

```
In [1]: import re

import pandas as pd
import spacy
from spacy.matcher import Matcher
from spacy import displacy
from IPython.display import HTML, display

nlp = spacy.load("en_core_web_sm")

def render_dep(doc):
    """Render a dependency parse that scales to the page width.

    displacy gives the SVG a fixed pixel width, which clips when the notebook is
    printed to PDF. I add a viewBox and set the width to 100% so the browser
    scales
    it to the container instead of overflowing."""
    svg = displacy.render(doc, style="dep", jupyter=False,
                          options={"compact": True, "distance": 90, "add_lemma":
False})
    w = int(re.search(r'width="(\d+)"', svg).group(1))
    h = float(re.search(r'height="([\d.]+)"', svg).group(1))
    svg = svg.replace(f'width="{w}"', 'width="100%"', 1)
    svg = svg.replace(f'height="{h}"', f'viewBox="0 0 {w} {h}"
preserveAspectRatio="xMinYMin meet"', 1)
    display(HTML(f'<div style="max-width:100%; overflow-x:auto;">{svg}</div>'))
```

```
print("spaCy version:", spacy.__version__)
print("pipeline components:", nlp.pipe_names)
```

spaCy version: 3.8.13

pipeline components: ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']

Task 1: Regex Pattern Drafting (international phone numbers)

I need a pattern that pulls international phone numbers out of free text. The brief gives three shapes to cover: `+1-555-0123`, `+(44)2079460958` and `0015551234567`. Reading across them, a valid number here is:

- an international prefix, either a literal `+` or the `00` trunk code,
- an optional country code, which may be wrapped in parentheses like `(44)`,
- then groups of digits joined by spaces, hyphens or dots (or no delimiter at all).

I anchor on the prefix because that is what makes a number "international" and it keeps the pattern from grabbing every random integer in the text. The delimiter class `[\s.\-]?` makes the separators optional and interchangeable so all three styles are accepted.

```
In [2]: phone_pattern = r"(?:\+|00)\d{1,4}\d{1,4}(?:[\s.\-]?\d{1,4})+"

sample_text = (
    "Call our US line on +1-555-0123, or the London office at +(44)2079460958. "
    "From abroad you can also dial 0015551234567. "
    "A dotted format like +49.30.123456 and a spaced one +81 3 1234 5678 are fine "
    "too. "
    "These are not phone numbers: order 12345, the year 2026, or the ratio 3.14."
)

matches = re.findall(phone_pattern, sample_text)
phone_df = pd.DataFrame({"matched_number": matches})
phone_df
```

Out[2]:

	matched_number
--	----------------

0	+1-555-0123
---	-------------

1	+(44)2079460958
---	-----------------

2	0015551234567
---	---------------

3	+49.30.123456
---	---------------

4	+81 3 1234 5678
---	-----------------

Every genuine international number is picked up, including the dotted and space separated variants, while the bare integers (`12345`, `2026`, `3.14`) are correctly ignored because they carry no `+` or `00` prefix.

I want to be honest about scope here. This regex matches the *shape* of an international number; it is not full validation. It does not check that a country code actually exists, nor that the digit count is legal for that country, and a determined edge case (an extension, a number split oddly across lines) could still slip through or be missed. Proper validation is what libraries like Google's `libphonenumber` exist for. For extracting candidates from text, which is the task, a shape based pattern anchored on the international prefix is the right level of effort.

Task 2: SpaCy Token Attributes

I process the brief's sentence and print `text`, `pos_`, `dep_` and `is_stop` for every token. A DataFrame is the clearest way to line these up.

```
In [3]: sentence = "Apple is looking at buying U.K. startups for $1 billion."
doc = nlp(sentence)

token_df = pd.DataFrame([
    {"text": t.text, "pos_": t.pos_, "dep_": t.dep_, "is_stop": t.is_stop}
    for t in doc
])
token_df
```

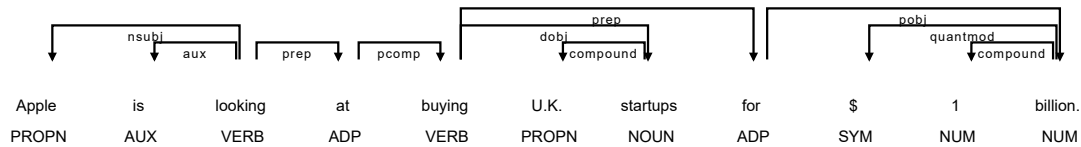
```
Out[3]:
```

	text	pos_	dep_	is_stop
0	Apple	PROPN	nsubj	False
1	is	AUX	aux	True
2	looking	VERB	ROOT	False
3	at	ADP	prep	True
4	buying	VERB	pcomp	False
5	U.K.	PROPN	compound	False
6	startups	NOUN	dobj	False
7	for	ADP	prep	True
8	\$	SYM	quantmod	False
9	1	NUM	compound	False
10	billion	NUM	pobj	False
11	.	PUNCT	punct	False

A few things stand out. `Apple` is tagged `PROPN` with the dependency `nsubj`, so spaCy has correctly read it as the subject of the sentence rather than the fruit. The tokenizer keeps `U.K.` as a single token instead of splitting on the full stops, which is the behaviour I want for an abbreviation. The money expression is split into `$` and `1` and `billion`, and the

function words (`is` , `at` , `for`) come back with `is_stop = True` , which is what a stop word filter would later remove. Below I render the dependency parse so the `dep_` column has a visual counterpart.

```
In [4]: render_dep(doc)
```



Task 3: Rule-based Matching (Golden Retriever)

The goal is a `Matcher` that finds "Golden Retriever" regardless of casing and also catches the hyphenated "Golden-Retriever". The casing part is easy: matching on the `LOWER` attribute makes the patterns case insensitive. The hyphen needs a little thought, because spaCy tokenizes `Golden-Retriever` into three tokens, `Golden` , `-` , `Retriever` , not two. So I add a second pattern with a punctuation token in the middle.

```
In [5]: matcher = Matcher(nlp.vocab)
matcher.add("GOLDEN_RETRIEVER", [
    [{"LOWER": "golden"}, {"LOWER": "retriever"}],          # spaced,
    any case
    [{"LOWER": "golden"}, {"IS_PUNCT": True}, {"LOWER": "retriever"}], #
    hyphenated form
])

doc3 = nlp(
    "I adopted a Golden Retriever last year. A golden retriever ran past me. "
    "The GOLDEN RETRIEVER at the show behaved perfectly. My neighbour's Golden-
    Retriever naps all day."
)

found = matcher(doc3)
match_df = pd.DataFrame([
    {"match": doc3[start:end].text, "start_token": start, "end_token": end}
    for _, start, end in found
])
match_df
```

Out[5]:

	match	start_token	end_token
0	Golden Retriever	3	5
1	golden retriever	9	11
2	GOLDEN RETRIEVER	16	18
3	Golden-Retriever	27	30

All four variants are caught: the standard casing, the lowercase version, the shouted uppercase one, and the hyphenated `Golden-Retriever`. Without the second pattern the hyphenated case would be missed entirely, because the `-` sits as its own token between the two words and breaks the adjacency that the first pattern relies on. This is the sort of tokenization detail that is easy to overlook until a matcher silently fails on real data.

Task 4: Named Entity Recognition (NER) Analysis

(a) and (b): extract entities and list their labels

```
In [6]: ner_sentence = "Elon Musk, the CEO of Tesla, joined a meeting in Berlin last Thursday."
doc4 = nlp(ner_sentence)

ent_df = pd.DataFrame([
    {"entity": e.text, "label": e.label_, "meaning": spacy.explain(e.label_)}
    for e in doc4.ents
])
ent_df
```

Out[6]:

	entity	label	meaning
0	Elon Musk	PERSON	People, including fictional
1	Tesla	ORG	Companies, agencies, institutions, etc.
2	Berlin	GPE	Countries, cities, states
3	last Thursday	DATE	Absolute or relative dates or periods

```
In [7]: # just the labels the model assigned, with a short gloss for each
for label in dict.fromkeys(e.label_ for e in doc4.ents):
    print(f"{label:8s} -> {spacy.explain(label)}")

displacy.render(doc4, style="ent", jupyter=True)
```

```
PERSON -> People, including fictional
ORG     -> Companies, agencies, institutions, etc.
GPE     -> Countries, cities, states
DATE    -> Absolute or relative dates or periods
```

Elon Musk **PERSON**, the CEO of Tesla **ORG**, joined a meeting in Berlin **GPE** last Thursday **DATE**.

spaCy tags `Elon Musk` as PERSON, `Tesla` as ORG and `Berlin` as GPE (a geopolitical entity), exactly the three the brief mentions, and it also picks up `last Thursday` as a DATE. It sensibly leaves `CEO` untagged, since a job title is not a named entity.

(c): a failure point on specialized medical or legal jargon

Pretrained models like `en_core_web_sm` are trained on general web and news text (OntoNotes), so their entity types are the everyday ones: PERSON, ORG, GPE, DATE and so on. They have never seen a domain vocabulary of drugs, diseases, statutes or case citations, and there is no label in the scheme for those concepts anyway. The result is that specialized terms get missed or, worse, confidently misclassified into the nearest general type. I will show this rather than just assert it.

```
In [8]: medical = nlp("The patient was prescribed 20mg of Lisinopril for hypertension and myocardial infarction.")
print("Medical sentence entities:")
for e in medical.ents:
    print(f" {e.text!r:20s} -> {e.label_}")

legal = nlp("The plaintiff cited 28 U.S.C. Section 1332 before the Ninth Circuit in Delaware.")
print("\nLegal sentence entities:")
for e in legal.ents:
    print(f" {e.text!r:20s} -> {e.label_}")
```

```
Medical sentence entities:
'20'                -> CARDINAL
'Lisinopril'        -> PERSON
```

```
Legal sentence entities:
'28'                -> CARDINAL
'the Ninth Circuit' -> ORG
'Delaware'          -> GPE
```

The medical sentence is the clearest example of the failure. `Lisinopril`, a common blood pressure drug, is labelled **PERSON**, presumably because it is a capitalised, name shaped word the model has never seen. The two conditions, `hypertension` and `myocardial infarction`, are not recognised as entities at all, and the dosage `20mg` is only partially caught as the bare number `20` tagged CARDINAL. The legal sentence shows the same pattern: the statute citation `28 U.S.C. Section 1332` is not understood as a legal reference, only the stray `28` surfaces as a CARDINAL, and while `the Ninth Circuit` scrapes into ORG, that is luck from its name shape rather than any understanding of courts. The practical lesson is that dropping a general purpose NER model onto medical or legal

text without fine tuning, or a domain model such as `scispaCy`, will produce quietly wrong entities, which in those fields is a real risk.

Task 5: Integration Task (hybrid Regex + dependency parsing)

The brief asks for a hybrid function: use a regex to find a custom serial number such as `SN-999-XYZ`, then use spaCy to confirm the serial sits inside a sentence that is a "Product Description", judged with dependency parsing.

The phrasing is a little open ended, so I will state my interpretation. The regex handles the lexical part, finding the serial's exact shape. spaCy then handles the structural part: I locate the sentence the serial falls in, and I use the dependency parse to decide whether that sentence is really describing a product. Concretely, I walk up the serial token's chain of syntactic heads (its `ancestors`) and treat the sentence as a product description if that chain reaches a head word whose lemma is `product` or `description`. That is a genuine use of the parse tree rather than a plain keyword search, because it requires the serial to be grammatically governed by the product-description head, not merely to co-occur with the word somewhere.

```
In [9]: SERIAL_RE = re.compile(r"SN-\d{3}-[A-Z]{3}")

def verify_serial_in_product_description(text):
    """Find serial numbers via regex, then use the dependency parse to check
    whether each one lives inside a product-description sentence."""
    doc = nlp(text)
    results = []
    for m in SERIAL_RE.finditer(text):
        span = doc.char_span(m.start(), m.end(), alignment_mode="expand")
        if span is None:
            continue
        sent = span.sent
        # dependency check: does the serial's head chain reach a
        # product/description head?
        ancestors = list(span.root.ancestors)
        governing = [span.root, *ancestors]
        is_product_desc = any(t.lemma_.lower() in {"product", "description"} for t
        in governing)
        dep_chain = " -> ".join(f"{t.text}({t.dep_})" for t in governing)
        results.append({
            "serial": m.group(),
            "in_product_description": is_product_desc,
            "sentence": sent.text.strip(),
            "dependency_path_to_root": dep_chain,
        })
    return results

demo = (
    "Product description: the DX200 unit ships with serial SN-999-XYZ and a"
```

```

printed manual. "
    "Our regional office is located in Berlin."
)
pd.DataFrame(verify_serial_in_product_description(demo))

```

```

Out[9]:

```

	serial	in_product_description	sentence	dependency_path_to_root
0	SN-999-XYZ	True	Product description: the DX200 unit ships with...	XYZ(pobj) -> with(pre) -> ships(appos) -> des...

The function finds `SN-999-XYZ` and confirms it belongs to the product description sentence. The dependency path is the interesting part: the serial token climbs through `with` (`prep`) and `ships` (`appos`) up to the sentence root, which is the word `description` itself. That is exactly the structural evidence I wanted, the serial is syntactically hanging off the product-description head, so the check passes for the right reason and not just because the word "product" happens to appear nearby.

```

In [10]: # a contrasting case: the same shape of serial, but in a sentence that is not a
product description
counter = "The invoice lists SN-100-ABC as a line item. This warranty covers one
year."
pd.DataFrame(verify_serial_in_product_description(counter))

```

```

Out[10]:

```

	serial	in_product_description	sentence	dependency_path_to_root
0	SN-100-ABC	False	The invoice lists SN-100-ABC as a line item.	ABC(dobj) -> lists(ROOT)

Here `SN-100-ABC` is still found by the regex, but the dependency check returns False: its head chain runs up to `lists`, an invoice sentence, with no product or description head in sight, so the function correctly declines to verify it. The two examples together show the hybrid working as intended, the regex is the fast lexical filter and the dependency parse is the structural confirmation on top of it.